



# Enhancing Programming Knowledge Tracing by Interacting Programming Skills and Student Code

Mengxia Zhu\*

Siqi Han\*

51195100046@stu.ecnu.edu.cn

sqhan@stu.ecnu.edu.cn

School of Data Science and  
Engineering, East China Normal  
University  
Shanghai, China

Peisen Yuan

peiseny@njau.edu.cn

College of Artificial Intelligence,  
Nanjing Agricultural University  
Nanjing, China

Xuesong Lu<sup>†</sup>

xslu@dase.ecnu.edu.cn

School of Data Science and  
Engineering, East China Normal  
University  
Shanghai, China

## ABSTRACT

Programming education has received extensive attention in recent years due to the increasing demand for programming ability in almost all industries. Educational institutions have widely employed online judges for programming training, which can help teachers automatically assess programming assignments by executing students' code with test cases. However, a more important teaching process with online judges should be to evaluate how students master each of the programming skills such as strings or pointers, so that teachers may give personalized feedback and help them proceed to the success more efficiently. Previous studies have adopted deep models of knowledge tracing to evaluate a student's mastery level of skills during the interaction with programming exercises. However, existing models generally follow the conventional assumption of knowledge tracing that each programming exercise requires only one skill, whereas in practice a programming exercise usually inspects the comprehensive use of multiple skills. Moreover, the feature of student code is often simply concatenated with other input features without the consideration of its relationship with the inspected programming skills. To bridge the gap, we propose a simple attention-based approach to learn from student code the features reflecting the multiple programming skills inspected by each programming exercise. In particular, we first use a program embedding method to obtain the representations of student code. Then we use the skill embeddings of each programming exercise to query the embeddings of student code and form an aggregated hidden state representing how the inspected skills are used in the student code. We combine the learned hidden state with DKT (Deep Knowledge Tracing), an LSTM (Long Short-Term Memory)-based

knowledge tracing model, and show the improvements over baseline model. We point out some possible directions to improve the current work.

## CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Applied computing** → **E-learning**; **Interactive learning environments**.

## KEYWORDS

programming education, knowledge tracing, code representation, attention mechanism, intelligent education

## ACM Reference Format:

Mengxia Zhu, Siqi Han, Peisen Yuan, and Xuesong Lu. 2022. Enhancing Programming Knowledge Tracing by Interacting Programming Skills and Student Code. In *LAK22: 12th International Learning Analytics and Knowledge Conference (LAK22)*, March 21–25, 2022, Online, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3506860.3506870>

## 1 INTRODUCTION

Knowledge Tracing (KT) is a task which models a student's cognition of knowledge concepts during the interaction with exercises [4]. Recent advances in the integration of deep learning techniques and knowledge tracing have brought unprecedentedly accurate models for the task and promoted the research of applying deep KT models in different disciplines such as second-language and STEM education [9, 20].

In programming education, KT models are used to evaluate a student's mastery level of programming skills. Recently, training systems such as online judges have facilitated the development of deep KT models since they accumulate large amounts of student-exercise interactions that are necessary for training the deep networks [16, 18]. However, two specific properties inherent in programming training are largely neglected by the current programming KT models. First, a programming exercise usually inspects multiple programming skills, yet existing models generally follow the convention of the classic KT task and assume each exercise contains only one skill, or need to build a model for each skill separately. This makes the existing models unsuitable for deployment in real programming education scenarios. Second, while the code submitted to each exercise may perfectly imply the mastery level of the required programming skills, existing models have not sufficiently explored how to represent the code and how the code uses

\*Both authors contributed equally to this research.

<sup>†</sup>Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

LAK22, March 21–25, 2022, Online, USA

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-9573-1/22/03...\$15.00

<https://doi.org/10.1145/3506860.3506870>

the required skills. The most common way of incorporating the code feature is to learn a code embedding and directly concatenate it with other input features such as exercise ID and the correctness of the code [16].

In this short paper, we present our preliminary study to tackle the aforementioned issues. At the core of our idea is to establish the interaction between the multiple programming skills in each exercise and the corresponding student code, so that the KT model can better capture from the code the specific patterns reflecting the use of the skills. To this end, we propose to use the attention network [17] to extract the code feature with the required skills, where the required skills are used as the queries and the code representations are used as the keys and the values. By establishing such interactions, each skill would try to search relevant patterns from the code representations and aggregate the code pattern information to form a hidden state, indicating how the skill is used in the code. Then, the hidden states pertaining to all required skills in an exercise are further averaged and concatenated with other features to form the input to the KT model.

For the preliminary evaluation, we utilize the DKT [14] model as the backbone structure, which uses an LSTM network, and apply the above method at the input layer. However, the skill-code interaction method could be easily integrated into other deep KT models such as Dynamic Key-Value Memory Networks (DKVMN) [21] and context-aware attentive networks (AKT) [6]. With the structure of DKT, we compare the proposed method for extracting code feature with the simple feature concatenation approach. Furthermore, we investigate three different code embedding methods for representing student code and compare their effects in the current setting. Preliminary results show that our proposed skill-code interaction solution steadily outperforms the baseline and the simple concatenation approach.

## 2 RELATED WORK

## 2.1 Programming Knowledge Tracing

The idea of knowledge tracing is first proposed in programming education, where Corbett and Anderson [4] try to model students' changing knowledge state during programming skill acquisition. They use a tutoring system for students to learn Lisp language and use the learning data to build the Bayesian Knowledge Tracing (BKT) model. The problem with BKT is it does not analyze student code and has to build an individual model for each programming skill. To incorporate the code information, Kasurinen et al. [10] add the predefined statistics of student code into the BKT model. However, each model can still model the acquisition of only one programming skill.

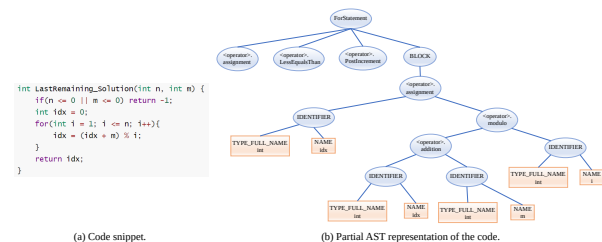
Recent studies have applied DKT [14] in programming education. For instance, Wang et al. [18] use the code embedding as the input to DKT. Their goal is to predict whether a student can solve the next exercise given the repeated code submissions to the current exercise, or can solve the current exercise at the next code submission. As such, their method is limited to model a student’s knowledge state for solving a single exercise, and cannot be directly used for open-ended knowledge tracing tasks, where students interact with many different exercises. To circumvent this problem, Swamy et al. [16] propose to compute the tf-idf score for each string token

in the source code and use the scores to form a vector as the code representation, which is concatenated with other exercise features to form the input. However, the length of the vector equals the size of entire vocabulary, making it impractical for training KT models on large code datasets, since the vocabulary will be too large and the code representation vector will be long and sparse, which consumes huge memory during training. Moreover, it follows the setting in BKT and models the mastery level of one knowledge concept at a time. Jiang et al. [7] propose to evaluate students' learning state based on code trajectory during their attempts on the same question. They calculate the tree edit distance between a student code and the optimal solution based on the AST, and predict the correctness of the student's submission on the next exercise using RNN or logistic regression. Other related works include [5, 8, 12].

## 2.2 Code Representation Learning

Student code embeddings are important features for programming knowledge tracing, because they capture the semantic property reflecting how well a student masters programming skills. With the rise of deep learning, code embeddings are recently obtained by training code representation learning models using deep networks.

The intermediate representation of source code, abstract syntax tree (AST), is frequently used as input for code representation learning in existing works, as an AST captures abundant structural information of the original source code that represents the semantic property. An example of partial AST<sup>1</sup> and the original code snippet is shown in Figure 1. Each node of the AST denotes a construct occurring in the source code. The string tokens in the source code are often organized as leaf nodes, and the syntactic relationships between them are organized as internal nodes.



**Figure 1: A simple C++ code snippet and the partial structure of the corresponding AST.**

Raychev et al. [15] convert source code into structural data by extracting structure information from the AST paths, and train a conditional random field model to predict the semantic property of source code. Inspired by this work, Alon et al. [2, 3] propose a series of methods to leverage the information contained in the AST paths and the corresponding neural network structures to learn the semantic features. Zhang et al. [22] split an AST into statement subtrees for representation learning. Other works [1, 13, 19] attempt to learn from the ASTs using more complicated neural networks such as tree-based CNNs (Convolutional Neural Networks), recursive neural networks and graph neural networks.

<sup>1</sup>The complete AST is too large to visualize in the draft. As such we just show the partial structure.

### 3 THE PROPOSED METHOD

We first introduce the overview of the proposed model in Section 3.1. Then we describe how we enable the interaction between the programming skills and the latent patterns of student code, and obtain the code embeddings using the attention mechanism in Section 3.2. Finally, we introduce three code representation learning methods for obtaining the latent code patterns in Section 3.3.

#### 3.1 Model Overview

The backbone structure used in the current work is the classical DKT model [14], which uses an LSTM network to model students' knowledge state during their interaction with exercises. In our specific model, the input information at each step  $t$  is a four-tuple  $(e_t, s_t, c_t, a_t)$ , where  $e_t$  is the exercise ID,  $s_t$  is a set of programming skills inspected by the exercise,  $c_t$  is the code submitted to the exercise by a student, and  $a_t$  is the correctness of the code, i.e., whether the code passes all test cases of the exercise.

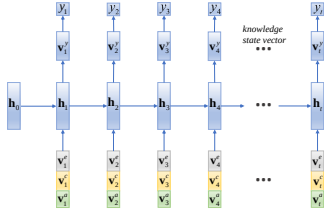


Figure 2: The main structure of the model.

To feed above information into the model, we extract three types of features as input. At each step  $t$ , we first use a bit vector  $\mathbf{v}_t^e \in \mathbb{1}^N$  to represent  $e_t$ , where  $N$  is the total number of skills, and the corresponding position of  $\mathbf{v}_t^e$  is set to 1 if the skill is required by  $e_t$ . Then we use a one-hot vector  $\mathbf{v}_t^a \in \mathbb{1}^2$  to represent  $a_t$ . Following previous studies [14, 21], if the code is incorrect,  $\mathbf{v}_t^a = [1, 0]$ ; otherwise,  $\mathbf{v}_t^a = [0, 1]$ . Finally, we use the attention mechanism to interact  $s_t$  with  $c_t$  and obtain the skill-code interaction output  $\mathbf{v}_t^c \in \mathbb{R}^{d_c}$ , where  $d_c$  is a hyperparameter. Generally speaking,  $\mathbf{v}_t^c$  is a code embedding representing whether and how the multiple skills in  $s_t$  are applied in the code  $c_t$ . The detailed method for obtaining  $\mathbf{v}_t^c$  is described in Section 3.2. The three feature vectors are concatenated as the input to the DKT model. The output is a latent knowledge state vector  $\mathbf{v}_t^y \in \mathbb{R}^N$ , where each element represents the probability of mastering the corresponding programming skill. The overall architecture is depicted in Figure 2. The computation of step  $t$  can be expressed as follows:

$$\mathbf{v}_t^x = \mathbf{v}_t^e \oplus \mathbf{v}_t^c \oplus \mathbf{v}_t^a, \quad (1)$$

$$\mathbf{h}_t = \mathbf{W}_{xh} \mathbf{v}_t^x + \mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{b}_h, \quad (2)$$

$$\mathbf{v}_t^y = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y, \quad (3)$$

where  $\oplus$  is the concatenating operation,  $\mathbf{W}_{xh}$ ,  $\mathbf{W}_{hh}$ ,  $\mathbf{W}_{hy}$  are the weight matrices, and  $\mathbf{b}_h$  and  $\mathbf{b}_y$  are the bias terms.

To train the model, we first obtain the output  $y_t$  at step  $t$ , which represents the probability that the code submitted to the next exercise  $e_{t+1}$  is correct. The probability is simply calculated as the average mastery probability of the skills required by  $e_{t+1}$ , i.e., the

skills in  $s_{t+1}$ . The mastery probability of each skill is indicated by the corresponding element in  $\mathbf{v}_t^y$ . The process can be formulated as follows:

$$y_t = \sigma\left(\frac{\text{Sum}(\mathbf{v}_t^y \odot \mathbf{v}_{t+1}^e)}{|s_{t+1}|}\right), \quad (4)$$

where  $\sigma$  is the Sigmoid function and  $\odot$  indicates Hadamard product. The function  $\text{Sum}()$  calculates the sum of a vector.

The loss function is the sum of cross entropy between the prediction  $y_t$  and the ground-truth correctness  $a_t$  of the code at each step, which is formulated as:

$$\mathcal{L} = - \sum_t (a_t \log y_t + (1 - a_t) \log(1 - y_t)). \quad (5)$$

#### 3.2 Code Embedding by Interacting Skills with Code

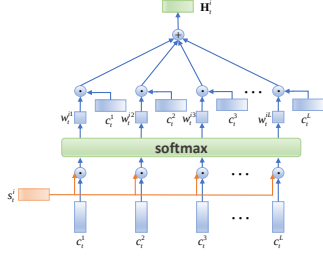
Given the set of programming skills  $s_t$  required by  $e_t$  and the submitted code  $c_t$ , the problem is to obtain a code embedding  $\mathbf{v}_t^c \in \mathbb{R}^{d_c}$  that captures the information of how  $c_t$  uses the skills in  $s_t$ . We transform this problem to the procedure of searching specific latent patterns in  $c_t$  relevant to the skills in  $s_t$ , which can be solved using the attention mechanism. In particular, the skills are used as the queries and the latent patterns of code are considered as the keys and the values. Then each skill produces a hidden state that aggregates the latent patterns of the code relevant to the skill through attention. Finally,  $\mathbf{v}_t^c$  is calculated as the average of the hidden states of all skills in  $s_t$ .

To obtain the skill embeddings, we first use a one-hot vector of length  $N$  to represent each skill<sup>2</sup> and form a matrix  $\mathbf{M}_t \in \mathbb{R}^{N \times N}$  to represent  $s_t$ , such that column  $i$  is the corresponding one-hot vector if the  $i^{\text{th}}$  skill is contained in  $s_t$  and is an all-zero vector otherwise, for  $i \in \{1, 2, \dots, N\}$ . Then we initialize a random embedding matrix  $\mathbf{E}_t \in \mathbb{R}^{d_c \times N}$  and obtain the query matrix  $\mathbf{Q}_t$  by  $(\mathbf{E}_t \times \mathbf{M}_t)^T$ , where the  $i^{\text{th}}$  row in  $\mathbf{Q}_t$  is the embedding of the  $i^{\text{th}}$  skill and the embeddings are trained with the model.

On the other hand, the latent patterns of a code that are relevant to the use of programming skills can be extracted from the structural information of the code, most commonly represented by AST [1, 3, 22]. Then we can train a set of vectors to learn the structural information, where each vector may capture a particular sub-structural pattern of the AST that is relevant to a programming skill. We refer to these vectors as “pattern vectors”. In Section 3.3, we describe three recently proposed AST-based methods for obtaining the pattern vectors. For each of the methods, we randomly select and stack  $L$  pattern vectors and form the key matrix  $\mathbf{K}_t \in \mathbb{R}^{L \times d_c}$  and value matrix  $\mathbf{V}_t \in \mathbb{R}^{L \times d_c}$ , where  $d_c$  is the vector length after embedding. Then we can use the attention mechanism to enable interaction between skill embeddings and pattern vectors, where each skill produces a hidden state aggregating the latent patterns of the code relevant to the skill. Particularly, the matrix of hidden states  $\mathbf{H}_t \in \mathbb{R}^{N \times d_c}$  of all skills is calculated as:

$$\mathbf{H}_t = \text{softmax}\left(\frac{\mathbf{Q}_t \times \mathbf{K}_t^T}{\sqrt{d_c}}\right) \times \mathbf{V}_t, \quad (6)$$

<sup>2</sup>The  $i^{\text{th}}$  element is set to 1 for the  $i^{\text{th}}$  skill.



**Figure 3: The skill-code interaction module.**  $s_t^i$  indicates the  $i^{th}$  skill,  $i \in \{1, 2, \dots, N\}$ .  $c_t^j$  indicates the  $j^{th}$  code pattern,  $j \in \{1, 2, \dots, L\}$ .  $w_t^{ij}$  is the attention weight of the  $i^{th}$  skill to the  $j^{th}$  code pattern. Finally,  $H_t^i$  is the hidden state capturing the code pattern information of the  $i^{th}$  skill.

where the  $i^{th}$  row in  $H_t$  captures the code pattern information of the  $i^{th}$  skill. Figure 3 shows how we obtain the  $i^{th}$  row  $H_t^i$  of  $H_t$ .

Finally, we multiply  $H_t$  with  $M_t$  to fetch the hidden states corresponding to the skills required by the exercise  $e_t$ . The code embedding  $v_t^c$  is the average of the fetched hidden states, which is calculated as:

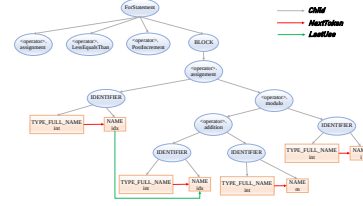
$$v_t^c = \frac{\text{Sum}(H_t^T \times M_t)}{|s_t|}, \quad (7)$$

where the function  $\text{Sum}()$  sums up all the columns in the matrix.

### 3.3 Code Representation Learning for Obtaining Pattern Vectors

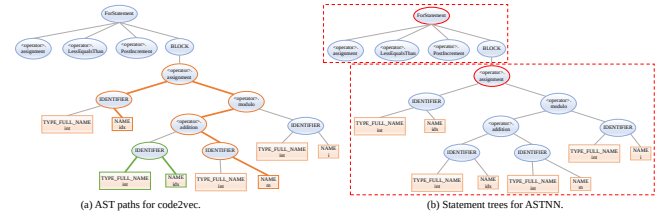
The remaining problem in Section 3.2 is how to obtain the pattern vectors that represent the sub-structure information of a code. A recent research hotspot is to train deep neural networks to learn the structural information from the AST of a code. And the trained models usually produce a set of embeddings that capture the sub-structures. This process is referred to as code representation learning, and the generated embeddings can be used as pattern vectors. As such, we introduce three recently proposed code representation learning methods that leverage ASTs for structural information extraction, namely, GGNN [1], code2vec [3] and ASTNN [22].

The three methods used in the current study embed a code snippet based on different types of sub-structures in the AST. In particular, GGNN [1] adds semantic edges such as Child, NextToken and LastUse between the AST nodes and forms a semantic graph. Then it adopts the gated graph neural networks [11] on the semantic graph to learn the embeddings of the nodes. As such the node embeddings can be used as the pattern vectors to represent the sub-structural information of the code. Figure 4 shows an example of the constructed graph of the AST in Figure 1. We include the aforementioned three types of edges. The second method, code2vec [3], extracts the AST paths between the leaf nodes. For each path, it concatenates the embeddings of the token of start node, the internal path, and the token of end node to form a path embedding. Then code2vec aggregates a set of path embeddings for code representation learning. As such the path embeddings of a code snippet can be used as the pattern vectors to represent the sub-structural information of the code. Figure 5(a) shows two paths in the AST, which



**Figure 4: The directed graph for GGNN.**

are colored in orange and green, respectively. The third method, ASTNN [22], embeds the sub-trees corresponding to the statements in the source code. Therefore, the embeddings of sub-trees can be used as the pattern vectors to represent the sub-structural information of the corresponding code. Figure 5(b) shows two statement trees of the AST in the red rectangles.



**Figure 5: Sub-structures for code2vec and ASTNN.**

To summary, we use three AST-based code representation learning methods to obtain the pattern vectors that capture the sub-structural information of a code, on the level of AST nodes, AST paths and AST sub-trees, respectively. We compare their performance when interacting with the skill embeddings. For each method, we randomly select a fixed number of the generated pattern vectors and input them to the model for end-to-end learning.

## 4 PERFORMANCE EVALUATION

### 4.1 The Dataset

Existing publicly available student program datasets either contain very few exercises or don't have ordered sequences of student-exercise interactions with sufficient length. As such we collect a student-exercise interaction dataset from Codeforces<sup>3</sup>, an open online judge for programming training and competition hosted by a group from the Saratov State University. We randomly select 300 users that had made at least 10 code submissions at the time of data collection, and crawl their complete submission history. After necessary cleaning such as removing non-compileable code snippets and duplicated submissions, we obtain 165,214 code submissions to a total of 7,152 programming exercises. Each submission contains the data of the timestamp, the required programming skills, the submitted code, the judging results and so on. The programming skills of an exercise are annotated by the users who design the exercise. The total number of the annotated skills in the dataset is 37. Example skills include strings, matrices, pointers and so on.

<sup>3</sup><https://codeforces.com/>

The dataset is randomly split into the training and testing set with proportion 4:1. We further split the training set into five folds for cross validation.

## 4.2 Evaluation Metric

As we predict the correctness of the code submitted at each step, we use AUC (Area Under the ROC Curve) as the evaluation metric, which are also used by previous works [14, 16, 18]. A higher AUC score means better prediction performance. The reported results are the average AUC with variation of five runs on the testing set.

## 4.3 Comparative Models

No previous programming knowledge tracing model targets at the open-ended scenario such that students interact with many different programming exercises. As such, in the preliminary study we mainly evaluate the effect of the proposed method for incorporating code features that reflect the use of multiple skills. To this end, we use DKT [14] as the baseline to evaluate the model performance without the code feature. Second, we use the three code representation learning methods to produce embeddings of code patterns (pattern vectors) and directly concatenate the generated code embeddings with other input features. The resulted models are named GGNN-concat, code2vec-concat and ASTNN-concat, respectively. Finally, we apply our attention technique to obtain the code embedding and concatenate it with other input features. The resulted models are named GGNN-attn, code2vec-attn and ASTNN-attn, respectively. All source code is available at <https://github.com/pkt-attn/pkt-attn>.

## 4.4 The Hyperparameters

The hyperparameters are obtained via cross-validation on the training set. For GGNN-concat and GGNN-attn, we set the number of graph nodes to 200 and the embedding dimension of nodes to 64. The learning rates are 0.01 and 0.005, and the regularization terms are 0.00001 and 0.0001. For code2vec-concat and code2vec-attn, we set the number of randomly selected paths to 50 and 200. The embedding dimension of AST leaf nodes and tree paths and the dimension of the generated pattern vectors are set to 128 and 64. The learning rates are both 0.001, and the regularization terms are 0.00002 and 0.00025. For ASTNN-concat and ASTNN-attn, the number of randomly selected AST sub-trees is set to 10, and the embedding dimensions of sub-trees are set to 128 and 64. The learning rates are 0.005 and 0.001. The regularization terms are 0.0001 and 0.00015. The optimizer used in these models is Adam.

## 4.5 Preliminary Results

We report the preliminary results in Table 1. First, we observe that when adding the code feature, all models outperform the basic DKT model. Second, among the models using simple feature concatenation, ASTNN-concat achieves the best performance, indicating the simple concatenation method prefers complex sub-structures to represent code information. On the other hand, the best model among the attention-based methods is code2vec-attn, which indicates the AST paths may better reflect the information of the skills used in the code. However, the information needs to be extracted using carefully-designed methods such as attentions. The best model

**Table 1: Preliminary results.**

| MODEL           | AUC(%)               |
|-----------------|----------------------|
| DKT [14]        | 72.57±0.00026        |
| GGNN-concat     | 72.94±0.00076        |
| code2vec-concat | 73.01±0.00012        |
| ASTNN-concat    | 73.19±0.00024        |
| GGNN-attn       | <b>73.75±0.00007</b> |
| code2vec-attn   | <b>73.81±0.00013</b> |
| ASTNN-attn      | <b>73.75±0.00042</b> |

code2vec-attn improves DKT by 1.24 percentage points on AUC. Although the current improvements are not large, the extremely small variations indicate the improvements are statistically significant. In Section 5, we discuss several possible directions to improve the current work.

## 5 CONCLUSION AND FUTURE WORK

In this preliminary study, we investigate the problem of programming knowledge tracing in the open-ended scenario, where students interact with many different exercises and each exercise inspects multiple programming skills. We particularly study how to represent the code in the specific scenario and how to search the code patterns that reflect the programming skills required by an exercise. We use the attention mechanism to enable the search of relevant code patterns using the skills. We further investigate three different code representation learning methods for the generation of code patterns and compare their effects in the knowledge tracing task. The preliminary results show the attention-based methods steadily outperform the simple concatenation methods. Next, we will produce some qualitative examples to show what is actually learned by the attention mechanism to further strengthen the results. The proposed method can also be transferred to other domains, such as mapping the skills over features of students' activities or other forms of knowledge, as long as the features are properly represented so that they can be processed by the model.

In future, we plan to improve the current work on at least three aspects. First, in the current work, we connect the code representation learning model with the KT model and train the model in the end-to-end fashion. Therefore the training is very expensive and hard to converge. We plan to pretrain the code representations on some downstream tasks such as code classification and code clone detection [3, 19], and use the pretrained representations as input to the KT model. Also, the current code representation learning models are designed towards capturing the semantic property of a code. Instead, we will investigate better learning methods to capture the used skills in a code. Second, currently we simply use a bit vector to represent each exercise, which ignores the differences between the exercises with the same programming skills. Ideally, we should give the model the input embeddings for both the exercise and the skills. To achieve this, we may construct a graph that connects the exercises based on some similarity measurement, for instance, there can be an edge connecting two exercises if they share most

similar skills. Then we can apply graph neural networks or graph embedding methods to learn the embeddings of the exercises. The embeddings of exercises and skills are then integrated as model input. Last but not least, we will investigate more approaches to construct the entire model for programming knowledge tracing. For example, a recent study [6] has shown that the pure attention-based architecture outperforms the sequence-based architecture on the classic knowledge tracing task. This inspires us to design attention-based models for both code feature learning and knowledge tracing, and further promote the prediction performance.

## ACKNOWLEDGMENTS

This work was supported by the grants from the National Natural Science Foundation of China (Grant No. U1811264, U1711262 and 62072185), and by National Key R&D Program of China (2018YFC0831904).

## REFERENCES

- [1] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to Represent Programs with Graphs. In *International Conference on Learning Representations (ICLR)*. 1–17.
- [2] Uri Alon, Omer Levy, and Eran Yahav. 2019. code2seq: Generating Sequences from Structured Representations of Code. In *International Conference on Learning Representations (ICLR)*.
- [3] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2019. code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 1–29.
- [4] Albert T Corbett and John R Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction* 4, 4 (1994), 253–278.
- [5] Andrew Emerson, Michael Geden, Andy Smith, Eric Wiebe, Bradford Mott, Kristy Elizabeth Boyer, and James Lester. 2020. Predictive Student Modeling in Block-Based Programming Environments with Bayesian Hierarchical Models. In *Proceedings of the 28th ACM Conference on User Modeling, Adaptation and Personalization*. 62–70.
- [6] Aritra Ghosh, Neil Heffernan, and Andrew S Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. 2330–2339.
- [7] Bo Jiang, Simin Wu, Chengjiu Yin, and Haifeng Zhang. 2020. Knowledge Tracing Within Single Programming Practice Using Problem-Solving Process Data. *IEEE Transactions on Learning Technologies* 13, 4 (2020), 822–832.
- [8] Bo Jiang, Y Ye, and Haifeng Zhang. 2018. Knowledge tracing within single programming exercise using process data. In *Proceedings of the 26th International Conference on Computers in Education*. 89–94.
- [9] Masahiro Kaneko, Tomoyuki Kajiwara, and Mamoru Komachi. 2018. TMU system for SLAM-2018. In *Proceedings of the Thirteenth Workshop on Innovative Use of NLP for Building Educational Applications*. 365–369.
- [10] Jussi Kasurinen and Uolevi Nikula. 2009. Estimating programming knowledge with Bayesian knowledge tracing. *ACM SIGCSE Bulletin* 41, 3 (2009), 313–317.
- [11] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2015. Gated graph sequence neural networks. *arXiv preprint arXiv:1511.05493* (2015).
- [12] Ye Mao. 2019. One minute is enough: Early prediction of student success and event-level difficulty during novice programming tasks. In *Proceedings of the 12th International Conference on Educational Data Mining (EDM)*.
- [13] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional neural networks over tree structures for programming language processing. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 30.
- [14] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. 2015. Deep knowledge tracing. In *Proceedings of the 28th International Conference on Neural Information Processing Systems (NIPS)*, Vol. 1. 505–513.
- [15] Veselin Raychev, Martin Vechev, and Andreas Krause. 2015. Predicting program properties from “big code”. *ACM SIGPLAN Notices* 50, 1 (2015), 111–124.
- [16] Vinitra Swamy, Allen Guo, Samuel Lau, Wilton Wu, Madeline Wu, Zachary Pardos, and David Culler. 2018. Deep knowledge tracing for free-form student code progression. In *International Conference on Artificial Intelligence in Education (AIED)*. Springer, 348–352.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NIPS)*. 5998–6008.
- [18] Lisa Wang, Angela Sy, Larry Liu, and Chris Piech. 2017. Learning to Represent Student Knowledge on Programming Exercises Using Deep Learning. In *Proceedings of the 10th International Conference on Educational Data Mining (EDM)*. 324–329.
- [19] Martin White, Michele Tufano, Christopher Vendome, and Denys Poshyvanyk. 2016. Deep learning code fragments for code clone detection. In *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 87–98.
- [20] Chun-Kit Yeung and Dit-Yan Yeung. 2019. Incorporating features learned by an enhanced deep knowledge tracing model for stem/non-stem job prediction. *International Journal of Artificial Intelligence in Education (IJAIED)* 29, 3 (2019), 317–341.
- [21] Jiani Zhang, Xingjian Shi, Irwin King, and Dit-Yan Yeung. 2017. Dynamic key-value memory networks for knowledge tracing. In *Proceedings of the 26th international conference on World Wide Web (WWW)*. 765–774.
- [22] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. 2019. A novel neural source code representation based on abstract syntax tree. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 783–794.